

LitAgent System Documentation

Ashkan Helmi Tabrizi (K12455989)

Jujun Zhu (K12430630)

2026-06-14

Contents

Cover Page	2
Table of Contents	3
Goal of the System	4
Stakeholders	4
Requirements	5
Functional Requirements	5
Non-Functional Requirements	5
AI-Related Requirements	6
Use Case Descriptions	7
UC1: Generate Research Dossier	7
UC2: Retrieve Literature Metadata	7
SUC1: Synthesize Academic Text	7
SUC2: Validate JSON Schema	8
Use Case Diagram	9
Implemented Use Cases	10
Traceability Matrix	10
Implementation Extensions Beyond the Original Assignments	11
Domain Model	12
Architecture Diagram	13
Component Descriptions	14
Design Questions and Answers	14

Cover Page

System name: LitAgent

System type: Automated Information Retrieval and Academic Literature Synthesis

Team members:

Name	K-Number
Ashkan Helmi Tabrizi	K12455989
Jujun Zhu	K12430630

Repository: <https://github.com/Jujun111/LitAgent>

Prototype status: Implemented course prototype with deterministic LangGraph orchestration, Semantic Scholar retrieval, local llama.cpp Qwen3.5-9B VLM inference, schema validation, Streamlit dashboard, PDF layout ingestion, and fidelity evaluation tools.

Table of Contents

This document is exported with an automatic table of contents in the final PDF. The main structure is:

1. Goal of the System
2. Stakeholders
3. Requirements
4. Use Case Descriptions
5. Use Case Diagram
6. Implemented Use Cases
7. Traceability Matrix
8. Implementation Extensions Beyond the Original Assignments
9. Domain Model
10. Architecture Diagram
11. Component Descriptions
12. Design Questions and Answers

Goal of the System

LitAgent is a deterministic AI orchestration system for academic literature retrieval, analysis, and synthesis. Its main goal is to help academic researchers and students submit a research topic and receive a structured research dossier that summarizes relevant papers, highlights key findings, reports limitations, and includes an AI-generated-content disclaimer.

Unlike a simple LLM wrapper, LitAgent separates deterministic control flow from AI generation. The non-AI controller decides when to retrieve papers, chunk text, build prompts, call a local model service, validate JSON, retry invalid outputs, and present the final dossier. The local model is only used for constrained generation and extraction, not for freely controlling the application workflow.

The implemented prototype supports:

- Metadata and abstract retrieval from Semantic Scholar.
- Mock-paper mode for demos without network or API access.
- LangGraph finite-state-machine orchestration.
- Local OpenAI-compatible llama.cpp inference using Qwen3.5-9B VLM.
- Pydantic validation of **ResearchDossier** JSON.
- Retry behavior for invalid model outputs.
- Streamlit dashboard output.
- Optional full-paper PDF layout ingestion through Docling.
- Optional pixel-level figure/table analysis through Qwen3.5 VLM image input.
- Reproducible benchmark and fidelity evaluation scripts.

This final document preserves the requirement and use-case numbering from Assignment 2 and Assignment 3. Later implementation work added PDF ingestion, pixel-level vision, benchmark scripts, and llama.cpp serving; those additions are documented as prototype extensions rather than new baseline requirement IDs or new baseline use-case IDs.

Stakeholders

Stakeholder	Interest
Academic researchers	Need quick, structured summaries of relevant research literature.
University students	Need support for exploring a research topic and identifying key papers.
Course instructors	Need a prototype that demonstrates requirements engineering, architecture, AI components, and implemented use cases.
Academic publishers	Their metadata and papers may be queried and summarized, so access limits and attribution matter.
Original authors	Their research should be represented accurately and with clear AI-output limitations.
Intellectual property and copyright entities	Care about fair use, data access, and not redistributing copyrighted full papers.

Requirements

Functional Requirements

ID	Requirement	Implemented Status
Req01	When the user submits a research query, LitAgent shall invoke the Semantic Scholar API to retrieve a maximum of 10 relevant academic paper metadata records.	Implemented for live mode; mock and fixed benchmark modes are available as explicit prototype aids.
Req02	While executing the retrieval pipeline, LitAgent shall enforce state transitions strictly through a predefined deterministic Finite State Machine (FSM).	Implemented with LangGraph nodes, edges, and conditional retry routing.
Req03	When the full texts of the academic papers are successfully downloaded, LitAgent shall prompt the Summarization Agent to generate a synthesis of the findings.	Implemented for retrieved abstracts, fixed benchmark records, and PDF-derived chunks. The current prototype does not automatically download every full paper from external links by default.
Req04	When the Summarization Agent generates the synthesis, LitAgent shall enforce a strict Pydantic-defined JSON schema on the model's output to guarantee valid syntax.	Implemented with the ResearchDossier Pydantic model and schema validation.
Req05	If the generated synthesis is successfully validated against the JSON schema, LitAgent shall present the compiled research dossier to the user on the dashboard.	Implemented in Streamlit with dossier, raw JSON, prompt, papers, chunks, visuals, and source tabs.
Req06	If the Summarization Agent fails to output valid JSON after three retry attempts, LitAgent shall terminate the agent loop and return a standardized error message to the user.	Implemented through retry counters, validation attempts, and controlled error reporting.

Non-Functional Requirements

ID	Requirement	Implemented Status
NfReq01	Semantic Scholar usage should respect a 100 requests per 5 minutes limit.	Implemented with SemanticScholarRateLimiter .
NfReq02	The prototype shall be implemented in Python and use LangGraph for deterministic orchestration.	Implemented.

ID	Requirement	Implemented Status
NfReq03	When the Summarization Agent is invoked, LitAgent shall generate the final JSON-formatted synthesis in less than 60 seconds per paper to ensure a responsive user experience.	Implemented with timeout configuration, latency metrics, and prompt-size controls; actual latency depends on local hardware and model settings.
NfReq04	When extracting factual claims from the source text, the Summarization Agent shall achieve an extraction fidelity of at least 95% without introducing external, unverified information.	Evaluated for the text-only benchmark; full multimodal fidelity remains a prototype extension and is not claimed at 95%.
NfReq05	AI-generated results shall include a disclaimer.	Implemented with <code>AI_DISCLAIMER</code> .

AI-Related Requirements

The AI-related requirements from Assignment 2 are Req03, Req04, NfReq03, NfReq04, and NfReq05. The final implementation satisfies these requirements through a local llama.cpp OpenAI-compatible model service, source-grounded prompting, Pydantic validation, retry handling, latency tracking, and an explicit AI-output disclaimer.

Use Case Descriptions

UC1: Generate Research Dossier

Primary actor: Academic Researcher

Description: The main FSM loop aggregates papers and presents the final dossier.

Main success scenario:

1. Researcher submits a query.
2. System executes UC2 (Retrieve Literature Metadata).
3. System executes SUC1 (Synthesize Academic Text) for each paper or selected source bundle.
4. System executes SUC2 (Validate JSON Schema) on the AI output.
5. System presents the compiled dossier with an AI-generation disclaimer.

Exception scenarios:

- UC2 returns a failure, such as an unreachable API. The system alerts the user and halts execution or uses explicit mock fallback if that demo option is enabled.
- The local model server is not running. The UI reports how to start llama.cpp.
- JSON validation fails. The FSM retries SUC1 up to three times, then reports failure.

UC2: Retrieve Literature Metadata

Primary actor: LitAgent Controller

Secondary actor: Semantic Scholar API

Description: The system retrieves up to 10 relevant papers for a submitted query.

Main success scenario:

1. The controller sends a query to Semantic Scholar.
2. The rate limiter controls request frequency.
3. Semantic Scholar returns metadata and available full-text links.
4. The system normalizes the records into internal paper dictionaries and source chunks.

Alternative scenarios:

- Mock mode returns deterministic demo papers.
- Fixed benchmark mode loads pre-labeled source papers.
- If the 100-request/5-minute rate limit is approaching, the system introduces a programmatic delay.

Exception scenarios:

- Semantic Scholar returns HTTP 429 or 500. The system logs the error and aborts retrieval unless explicit fallback is enabled.

SUC1: Synthesize Academic Text

Primary actor: Internal AI component

Description: Local LLM ingestion and generation.

Main success scenario:

1. System chunks the downloaded or selected source text.
2. Qwen3.5-9B VLM processes the text through the local llama.cpp OpenAI-compatible service.
3. Qwen3.5 returns a raw JSON string intended to satisfy the `ResearchDossier` schema.

Exception scenarios:

- Local hardware latency exceeds the configured timeout. The inference process is terminated to prevent hanging.
- The model output is truncated or invalid. SUC2 rejects it and the FSM retries.
- The model server is unreachable. The UI displays a model-server startup hint.

SUC2: Validate JSON Schema

Primary actor: LitAgent Controller

Description: Structural validation of the LLM output.

Main success scenario:

1. Pydantic parses the raw model output.
2. The string passes validation and is mapped to a **ResearchDossier** data object.

Exception scenario:

- The string fails validation. The system retries SUC1 up to three times.
- After three failed retries, the system gracefully terminates the node and returns an error flag.

Use Case Diagram

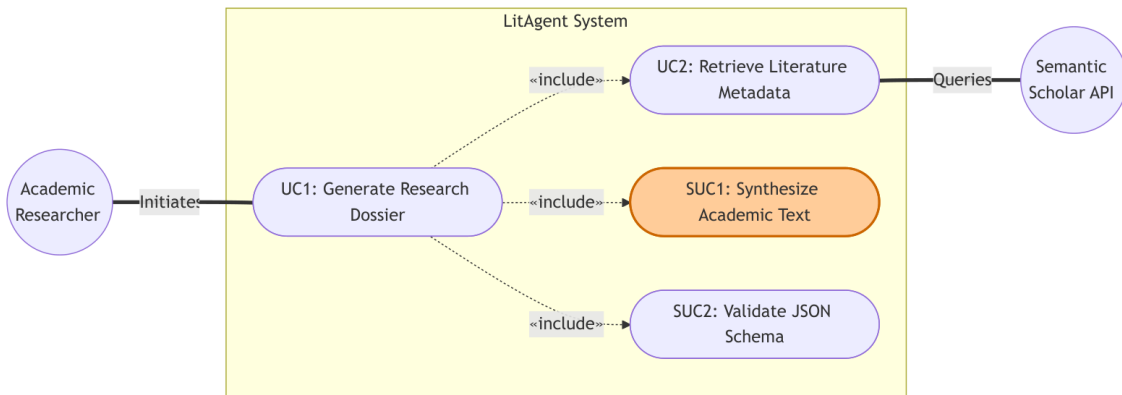


Figure 1: Use case diagram

Implemented Use Cases

Use Case	Implemented?	Evidence in Source Code
UC1 Generate Research Dossier	Yes	<code>run_pipeline</code> , <code>build_litagent_graph</code> , Streamlit Generate button
UC2 Retrieve Literature Metadata	Yes	<code>SemanticScholarClient</code> , <code>fetch_data_node</code> , live/mock/fixed modes
SUC1 Synthesize Academic Text	Yes	<code>call_openai_compatible_node</code> , prompt builder, llama.cpp scripts
SUC2 Validate JSON Schema	Yes	<code>ResearchDossier</code> , <code>validate_json_node</code> , retry routing
Full open-access PDF download from Semantic Scholar links	Partial	Live metadata links are captured; automated full-paper download is not the default path
95% full-paper multimodal fidelity	Partial	Text-only benchmark exceeds 95%; pixel benchmark is smaller and targets 80% v1 recall

All four baseline use cases from Assignment 3 are implemented in the source code. The implemented prototype includes the required AI components and follows the described deterministic architecture. Additional dashboard, PDF, vision, mock, and benchmark capabilities are implementation extensions rather than new baseline use cases.

Traceability Matrix

Requirement	Use Case(s)	Component(s)	Implementation Evidence
Req01 Retrieve up to 10 papers	UC2	Semantic Scholar Client	live retrieval node
Req02 FSM orchestration	All baseline use cases	LangGraph Controller	graph nodes and edges
Req03 Prompt AI when texts are ready	UC1, SUC1	Prompt Builder, llama.cpp Service	prompt builder and LLM call node
Req04 Strict JSON schema	SUC2	Pydantic Validator	<code>ResearchDossier</code> schema
Req05 Present dossier on dashboard	UC1	Streamlit UI	<code>streamlit_app.py</code>
Req06 Terminate after 3 invalid JSON attempts	SUC2	LangGraph Controller	validation retry route
NfReq01 Rate limiting	UC2	Rate Limiter	<code>SemanticScholarRateLimiter</code>
NfReq02 Python and LangGraph implementation	All baseline use cases	Backend	Python modules and LangGraph orchestration

Requirement	Use Case(s)	Component(s)	Implementation Evidence
NfReq03 60-second latency target	SUC1	Backend Metrics	latency metrics and timeouts
NfReq04 95% extraction fidelity target	SUC1	Evaluation Scripts	fidelity evaluator and gold data
NfReq05 AI disclaimer	UC1	Dossier Schema/UI	AI_DISCLAIMER rendered in dossier

Implementation Extensions Beyond the Original Assignments

The following features were added after the original Assignment 2 and Assignment 3 baselines. They are useful for the final prototype but are intentionally not renumbered as new formal requirements or use cases.

Mock-paper and mock-LLM demo path: Extends UC1, UC2, and SUC1. Implemented through mock paper records, mock LLM output, and `smoke_test.py`.

Fixed benchmark data loading: Extends UC1, SUC1, SUC2, and NfReq04. Implemented through `benchmarks/` data and `evaluate_fidelity.py`.

PDF layout ingestion: Extends Req03, UC1, and SUC1. Implemented as a prototype through `pdf_ingest.py`, `smoke_pdf_ingest.py`, and the Streamlit PDF controls.

Pixel-level visual extraction: Extends Req03, SUC1, and NfReq04. Implemented as an optional prototype through `vision_ingest.py`, `smoke_pixel_vision.py`, and the llama.cpp VLM service.

Expanded dashboard inspection tabs: Extends Req05 and UC1. Implemented in Streamlit through dossier, raw JSON, prompt, papers, chunks, visuals, and sources tabs.

llama.cpp replacing the earlier vLLM proposal: Extends NfReq03, NfReq04, and the Assignment 4 architecture design. Implemented through the `llm_service/` llama.cpp scripts and smoke checks.

The architecture evolved from the Assignment 4 vLLM proposal to llama.cpp because the selected GGUF/VLM model path was more practical for local Windows/WSL demonstration. The controller interface remained OpenAI-compatible, so the design principle of keeping the deterministic controller decoupled from the concrete model server was preserved.

Domain Model

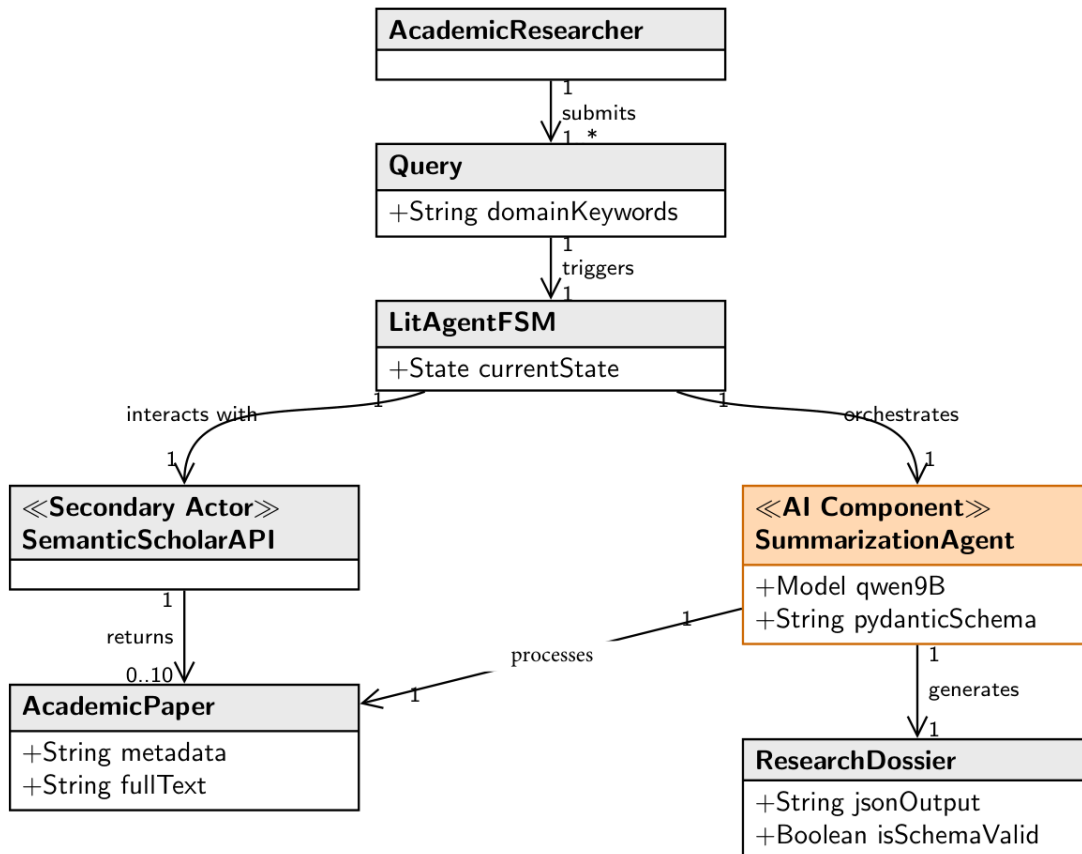


Figure 2: Domain model

Architecture Diagram

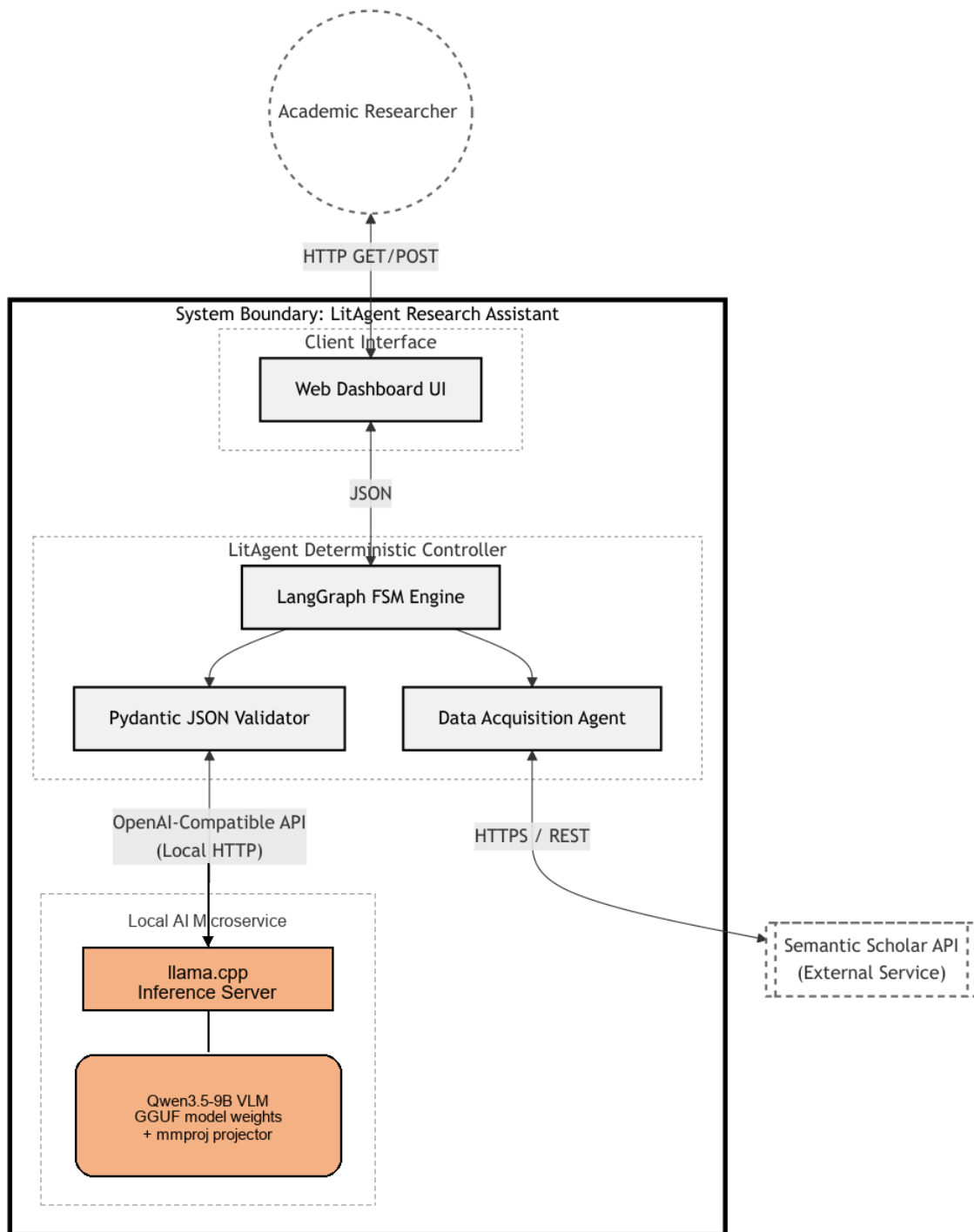


Figure 3: Architecture diagram

Component Descriptions

Component	Responsibility	Main Files
Streamlit Dashboard	User interface for selecting provider, entering query, uploading PDFs, running the pipeline, and inspecting results.	<code>streamlit_app.py</code>
LangGraph FSM Controller	Deterministic orchestration of fetch, vision extraction, chunking, prompting, LLM call, validation, and retry.	<code>litagent_backend.py</code>
Semantic Scholar Client	Live literature metadata retrieval and API rate limiting.	<code>litagent_backend.py</code>
Prompt Builder and Chunk Manager	Converts abstracts, PDF text, tables, captions, and vision observations into source-grounded prompt chunks.	<code>litagent_backend.py</code>
Local AI Microservice	Serves Qwen3.5-9B VLM through llama.cpp using an OpenAI-compatible HTTP API.	<code>llm_service/</code> scripts
Pydantic Validator	Enforces <code>ResearchDossier</code> structure, detects invalid JSON, and supports retry logic.	<code>litagent_backend.py</code>
PDF Ingestion	Converts full-paper PDFs into structured paper records with chunks, tables, captions, figures, and source references.	<code>pdf_ingest.py</code>
Vision Ingestion	Sends image crops to Qwen3.5 VLM and validates <code>VisionObservation</code> output.	<code>vision_ingest.py</code>
Evaluation Tools	Measures schema validity, required-fact recall, category recall, source trace rate, and latency.	<code>evaluate_fidelity.py</code> , benchmarks
CLI Demo	Small command-line demonstration path for the canonical backend.	<code>litagent_fsm.py</code>

Design Questions and Answers

1. How is the local Qwen model served without blocking the controller?

The model is served by llama.cpp as a separate OpenAI-compatible local HTTP service. The controller calls it through `POST /v1/chat/completions`, which keeps model execution outside the deterministic FSM code.

2. How decoupled is the controller from the specific model?

The controller only depends on an OpenAI-compatible API shape. The model alias, base URL, API key, and structured-output mode are configurable in Streamlit and scripts.

3. How does the system avoid brittle prompt-loop control?

The LLM never chooses control-flow actions. LangGraph defines the finite-state-machine transitions, and the LLM only returns structured data.

4. **How is JSON validity enforced?**
The prompt requests JSON, the llama.cpp request includes a schema-compatible response format, and Pydantic validates the returned `ResearchDossier`. Invalid JSON triggers retry.
5. **How does the system handle API rate limits?**
`SemanticScholarRateLimiter` tracks request timestamps and waits when the configured 100 requests per 5 minutes window is approached.
6. **How does the dashboard show reliability information?**
Streamlit displays fetch mode, LLM mode, FSM mode, JSON validity, validation attempts, latency, raw JSON, chunks, and source traces.
7. **Why does the system include mock mode?**
Mock mode ensures the prototype can be demonstrated without network access, API keys, model serving, or rate-limit risk.
8. **How is full-paper understanding supported?**
Docling parses local PDFs into text, tables, captions, figure placeholders, page numbers, and `source_ref` metadata. These chunks enter the same deterministic pipeline as retrieved abstracts.
9. **How is pixel-level figure/table understanding supported?**
PDF crops are sent to Qwen3.5 VLM through llama.cpp. The model returns validated `VisionObservation` JSON, which is converted into `vision` evidence chunks.
10. **How is extraction fidelity evaluated?**
The system includes frozen benchmarks with gold required facts. The evaluator runs the pipeline and reports schema validity, required-fact recall, category recall, source trace rate, and latency.